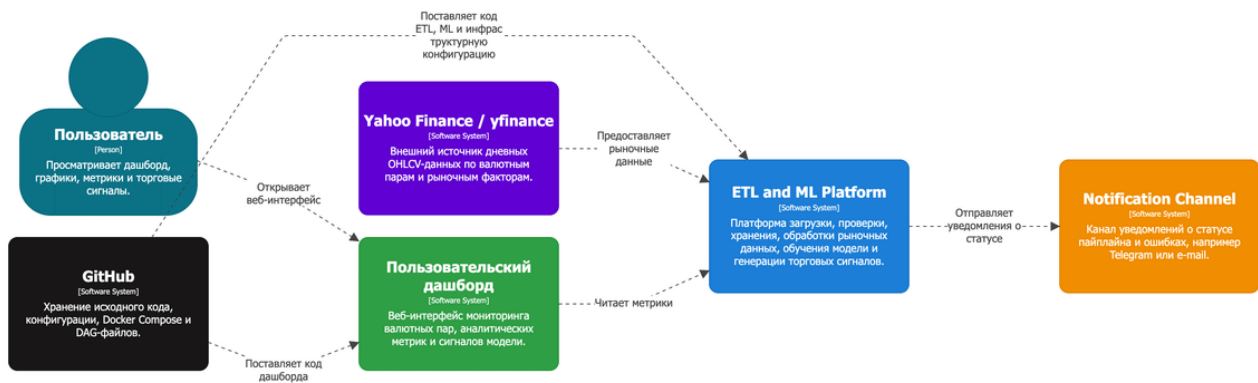


Архитектура конвейера данных для мониторинга валютных пар



System Landscape View

суббота, 4 апреля 2026 г. в 09:39 Москва, стандартное время

Архитектура проекта представляет собой полный прикладной конвейер для мониторинга валютных пар, который объединяет сбор данных, их проверку, хранение, аналитическую обработку, обучение модели машинного обучения, формирование торговых сигналов и пользовательскую визуализацию. Система спроектирована как набор взаимосвязанных подсистем, каждая из которых выполняет отдельную роль, но все вместе образуют единый поток данных от внешнего источника до конечного пользователя.

Внешним источником данных выступает Yahoo Finance через библиотеку yfinance. Из этого источника ежедневно извлекаются суточные OHLCV-данные по целевым валютным парам и связанным с ними контекстным рыночным активам. К таким активам относятся фондовые индексы, сырьевые инструменты, ETF, доходности облигаций и другие факторы, которые могут использоваться как в мониторинге, так и при построении модели. Источник данных не входит внутрь системы, а является внешней зависимостью, откуда конвейер получает входной поток рыночной информации.

Центральной подсистемой является ETL and ML Platform. Это серверная часть проекта, отвечающая за весь фоновый цикл обработки данных. Внутри этой платформы основной механизм оркестрации реализован через Apache Airflow. Airflow выбран как инструмент управления расписанием и последовательностью задач, поскольку он позволяет описывать процесс в виде DAG, запускать его регулярно, отслеживать статус выполнения и фиксировать ошибки. В архитектуре Airflow разделен на несколько контейнеров. Airflow Webserver обеспечивает интерфейс управления и контроля DAG-процессов. Airflow Scheduler отвечает за запуск задач по расписанию. Airflow Worker выполняет сами вычислительные шаги пайплайна.

Внутри Airflow Worker размещена логика ETL и ML в виде последовательности задач. Первая задача, Extract from yfinance, извлекает дневные OHLCV-данные по валютным парам и контекстным факторам. Вторая задача, Validate data, проверяет корректность структуры данных, наличие пропусков, типы значений и допустимость ценовых соотношений. После этого задача Load to PostgreSQL загружает очищенные данные в основную фактную таблицу базы данных. Следующий шаг, Calculate dashboard metrics, рассчитывает агрегированные показатели, которые нужны для пользовательского дашборда, например доходности и волатильность. Затем задача Build features формирует признаки для модели машинного обучения, используя исторические значения цен и информацию о связанных рыночных активах. После этого задача Train or refresh model обучает или обновляет модель классификации. Следующая задача, Generate predictions, строит прогноз на следующий день. Потом задача Write signals записывает предсказания и итоговые сигналы в таблицу сигналов. Финальная задача Notify and log status фиксирует результат выполнения в журнале и при необходимости отправляет уведомление во внешний канал.

Хранилищем данных служит PostgreSQL. Эта база данных является центральным репозиторием всей предметной и технологической информации. В архитектуре PostgreSQL выделен как отдельный контейнер, поскольку именно он обеспечивает долговременное хранение как сырых временных рядов, так и производных аналитических результатов. Внутри базы логически присутствует несколько таблиц. Таблица instruments является справочником всех инструментов системы. В ней содержатся как целевые валютные пары, так и контекстные рыночные активы. Таблица pair_context хранит связи между валютными парами и контекстными инструментами и задает факторное окружение каждой пары. Таблица ohlcv_daily является основной фактной таблицей, в которую загружаются ежедневные OHLCV-наблюдения. Таблица dashboard_metrics_daily представляет витрину рассчитанных показателей, которые затем используются в дашборде. Таблица features_daily предназначена для хранения построенных признаков, если архитектура предусматривает их материализацию в базе. Таблица model_signals_daily содержит результат работы модели, то есть прогнозы и торговые сигналы. Таблица model_registry хранит метаданные о версиях модели, времени обучения и параметрах, если требуется отслеживать жизненный цикл моделей. Таблица etl_log используется для журналирования статусов загрузки, ошибок и общей технической диагностики пайплайна. Отдельно могут использоваться служебные таблицы Airflow, связанные с исполнением DAG, но они относятся уже к техническому уровню поддержки оркестратора.

Для администрирования базы в архитектуру включен pgAdmin. Это вспомогательный контейнер, который используется разработчиком или администратором для просмотра структуры базы, настройки соединений, проверки содержимого таблиц и диагностики состояния PostgreSQL. Он не является частью пользовательского контура и не участвует в отображении данных конечному пользователю, но важен для эксплуатации и сопровождения проекта.

Подсистема пользовательского взаимодействия выделена отдельно и представлена как Пользовательский дашборд. Такой выбор архитектурно правильный, потому что пользователь не взаимодействует напрямую с ETL-конвейером, моделью или Airflow, а работает только с

готовым интерфейсом. Внутри этой подсистемы используется Nginx Reverse Proxy и Streamlit Dashboard. Nginx выполняет роль внешней точки входа и проксирует HTTP-запросы к приложению. Streamlit служит пользовательским веб-интерфейсом, в котором отображаются графики цен, аналитические показатели, сигналы модели и статус последней загрузки данных. Streamlit не занимается извлечением или преобразованием данных, а только читает уже подготовленные результаты из PostgreSQL. Благодаря этому пользовательский слой отделен от вычислительного и не создает дополнительной нагрузки на ETL-пайплайн.

Важной частью архитектуры является хранилище артефактов модели, обозначенное как Model Artifact Storage. Оно предназначено для сохранения сериализованных файлов обученной модели и связанных с ней артефактов. Это решение более правильно, чем хранение самой модели внутри таблиц базы данных. База данных используется для хранения метаданных, признаков и предсказаний, а бинарный артефакт модели сохраняется отдельно как файл. При обучении новая версия модели сохраняется в это хранилище, а при инференсе актуальная модель загружается из него для генерации прогнозов.

В архитектуре также предусмотрен внешний канал уведомлений. Он нужен для фиксации завершения пайплайна, ошибок загрузки или сбоев обучения модели. Это может быть Telegram, электронная почта или другой способ уведомления. Этот компонент усиливает практическую ценность проекта, поскольку превращает его из просто вычислительного стенда в наблюдаемую и управляемую систему.

GitHub в архитектуре играет роль внешней системы контроля версий и хранилища кода. В нем размещаются DAG-файлы Airflow, код дашборда Streamlit, Docker-конфигурация и прочие исходники проекта. GitHub не участвует в runtime-выполнении бизнес-логики, но важен как источник поставки кода и как инфраструктурный элемент, обеспечивающий воспроизводимость и сопровождение проекта.

Развертывание системы происходит в облачной инфраструктуре Timeweb Cloud на VPS-сервере, размещенном в Москве. Это deployment-уровень архитектуры. На VPS установлен Docker Engine, внутри которого запускаются все сервисные контейнеры. Такой подход удобен для БКР, потому что он показывает не только логическую, но и физическую сторону проекта. В контейнерах разворачиваются Nginx, Streamlit, Airflow Webserver, Airflow Scheduler, Airflow Worker, PostgreSQL, pgAdmin и хранилище артефактов модели. Таким образом, вся система развернута в едином изолированном окружении и может быть воспроизведена на другом сервере через Docker Compose.

Логика движения данных в архитектуре выглядит следующим образом. Сначала Airflow по расписанию инициирует извлечение данных из yfinance. Затем данные проходят стадию валидации. После успешной проверки они загружаются в PostgreSQL в таблицу ohlcv_daily. На основе этих данных рассчитываются показатели для витрины dashboard_metrics_daily. Из сырых временных рядов и контекстных факторов строятся признаки, которые используются для обучения или обновления ML-модели. После обучения модель сохраняется в хранилище

артефактов, а затем используется для генерации прогнозов на следующий день. Результаты инференса записываются в таблицу `model_signals_daily`. Статус выполнения и возможные ошибки пишутся в `etl_log` и, при необходимости, отправляются во внешний канал уведомлений. После этого Streamlit читает готовые метрики, сигналы и статусы из PostgreSQL и отображает их пользователю через веб-интерфейс.

С архитектурной точки зрения проект разделен на несколько уровней. Первый уровень — внешний, к нему относятся пользователь, GitHub, yfinance и канал уведомлений. Второй уровень — вычислительный, представленный ETL and ML Platform с Airflow и логикой обработки. Третий уровень — уровень хранения, который реализован через PostgreSQL и вспомогательный pgAdmin. Четвертый уровень — пользовательский, представленный подсистемой дашборда на базе Nginx и Streamlit. Пятый уровень — deployment-уровень, где все контейнеры физически размещены на VPS в Timeweb Cloud и управляются через Docker.

Преимущество такой архитектуры заключается в четком разделении ответственности между компонентами. Airflow отвечает только за оркестрацию и выполнение процесса. PostgreSQL отвечает за хранение данных и результатов. Streamlit отвечает только за визуализацию. Model Artifact Storage отвечает за файлы модели. GitHub отвечает за исходный код и версионирование. Timeweb Cloud и Docker обеспечивают физическое размещение и воспроизводимое развертывание. За счет такого разделения архитектура получается понятной, масштабируемой и логически завершенной.

С точки зрения масштабирования проект также построен с запасом. Можно добавлять новые валютные пары и контекстные активы без изменения общей логики, просто расширяя справочник `instruments` и таблицу `pair_context`. Можно увеличивать горизонт истории в `ohlcv_daily`, не меняя структуру витрин. Можно заменить модель классификации на другую, не меняя базовую схему пайплайна, так как `trainTask` и `inferenceTask` остаются теми же логическими узлами. Можно вынести хранилище моделей во внешний object storage. Можно усложнить дашборд, не затрагивая ETL. Можно добавить новые DAG-процессы в Airflow для расчета других признаков, бэктестов или мониторинга качества модели. Также архитектура допускает переход к более продвинутому хранению временных рядов, например к расширению TimescaleDB, если объем данных существенно вырастет.